1. Java Arrays: Meaning, Declaration & Creation

BCA study notes | Simple explanation + Java examples + diagrams

Meaning

*An **array** is a collection of values of the same data type stored under one variable name. Each value is called an **element**. Arrays have a fixed size.*

Why use arrays?

Instead of creating many variables such as mark1, mark2 and mark3, we can store all marks in one array and access them using an index.

Declaration

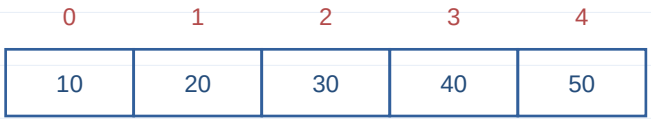
*Syntax: **dataType[] arrayName;** or **dataType arrayName[];**. The first style is commonly preferred.*

Creation

*The **new** keyword reserves memory for the array. The size is written inside square brackets.*

Java code example

```
int[] numbers;                  // declaration
numbers = new int[5];             // creation
int[] ages = new int[3];          // declaration + creation
```



Index is the position; first index is 0

2. Initialization & Default Values

BCA study notes | Simple explanation + Java examples + diagrams

Initialization

We can give values while creating an array, or assign values later using indexes.

Default values

When an array is created with **new**, Java gives default values: 0 for integer types, 0.0 for floating types, false for boolean, and null for reference types.

Important

Array size cannot be changed after creation. If you need a different size, create another array or use a collection such as ArrayList.

Java code example

```
int[] marks = {78, 85, 91};   // initialization
int[] scores = new int[3];
scores[0] = 70;
scores[1] = 80;
scores[2] = 90;
```

3. Length & Indexing

BCA study notes | Simple explanation + Java examples + diagrams

length property

Use **arrayName.length** to find the number of elements. It is a property, not a method, so do not write length().

Indexing

Array indexing starts from **0**. For an array of length 5, valid indexes are 0, 1, 2, 3 and 4.

Error

Accessing an invalid index causes **ArrayIndexOutOfBoundsException**. The last valid index is length - 1.

Java code example

```
int[] a = {10, 20, 30, 40, 50};
System.out.println(a.length); // 5
System.out.println(a[0]);     // 10
System.out.println(a[4]);     // 50
for (int i = 0; i < a.length; i++) {
    System.out.println(a[i]);
}
```

4. Multidimensional Arrays

BCA study notes | Simple explanation + Java examples + diagrams

Meaning

A multidimensional array is an array whose elements are also arrays. A two-dimensional array is often shown as rows and columns, like a table.

Declaration

Syntax: ***int[][] marks = new int[2][3];*** *means 2 rows and 3 columns.*

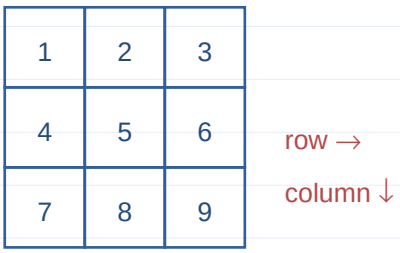
Access

Use two indexes: marks[row][column]. Row and column indexes both start at 0.

Java code example

```
int[][] data = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(data[1][2]); // 6
```

2D array diagram (3 rows × 3 columns)



5. Jagged & Anonymous Arrays

BCA study notes | Simple explanation + Java examples + diagrams

Jagged array

A jagged array is a 2D array in which each row can have a different length. First create the rows, then create each row separately.

Anonymous array

An anonymous array is created without giving it a variable name. It is useful when passing values directly to a method.

Remember

*Anonymous array syntax: **new int[]{4, 8, 12}**. Do not write a size inside the brackets when values are supplied.*

Java code example

```
// Jagged array
int[][] j = new int[3][];
j[0] = new int[]{1, 2, 3};
j[1] = new int[]{4, 5};
j[2] = new int[]{6, 7, 8, 9};

// Anonymous array passed to a method
printValues(new int[]{4, 8, 12});
```

Jagged array: each row has a different number of elements

6. Array of Objects

BCA study notes | Simple explanation + Java examples + diagrams

Meaning

An array of objects stores references to objects. Each element can refer to an object of a class.

Steps

First declare and create the reference array. Then create objects and assign them to array positions.

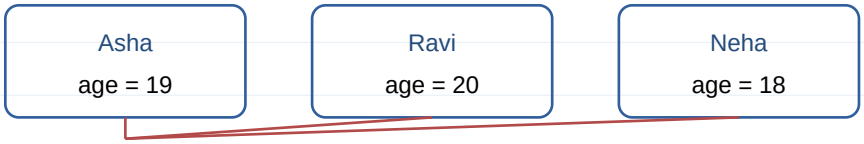
Important

Creating Student[3] creates three reference slots, not three Student objects. Each object must be created separately using new Student(...).

Java code example

```
class Student {
    String name;
    Student(String n) { name = n; }
}
Student[] list = new Student[2];
list[0] = new Student("Asha");
list[1] = new Student("Ravi");
```

Array of object references



7. java.util.Arrays

BCA study notes | Simple explanation + Java examples + diagrams

Purpose

The **java.util.Arrays** class provides ready-made methods to work with arrays. Import it using **import java.util.Arrays;**.

Useful methods

toString() prints a one-dimensional array neatly; sort() sorts values; binarySearch() searches a sorted array; equals() compares contents; copyOf() makes a copy.

Note

For nested arrays, use deepToString() and deepEquals() when you want to work with inner arrays too.

Java code example

```
import java.util.Arrays;
int[] a = {5, 2, 8};
System.out.println(Arrays.toString(a));
Arrays.sort(a);
System.out.println(Arrays.toString(a));
```

8. Sorting & Searching

BCA study notes | Simple explanation + Java examples + diagrams

Sorting

Sorting means arranging elements in ascending or descending order. Arrays.sort(array) sorts primitive values in ascending order.

Searching

Linear search checks each element one by one and works on unsorted arrays. Binary search repeatedly halves the search range and requires sorted data.

Binary search

Arrays.binarySearch(array, key) returns the index if found. If not found, it returns a negative value.

Java code example

```
int[] a = {9, 2, 6, 1};
Arrays.sort(a);                 // [1, 2, 6, 9]
int index = Arrays.binarySearch(a, 6);
System.out.println(index);      // 2
```

9. Copying & Comparing Arrays

BCA study notes | Simple explanation + Java examples + diagrams

Copying

Arrays.copyOf(original, newLength) creates a new array with the requested length. System.arraycopy() copies a range into an existing destination array.

Reference vs contents

Using == on arrays checks whether both variables refer to the same array object. Arrays.equals(a, b) checks whether corresponding elements are equal.

Nested arrays

Use Arrays.deepEquals() to compare nested arrays by their contents.

Java code example

```
int[] a = {2, 4, 6};
int[] b = Arrays.copyOf(a, a.length);
System.out.println(Arrays.equals(a, b)); // true
System.out.println(a == b);              // false
```

Copy creates a separate array

10. Quick Revision & Complete Example

BCA study notes | Simple explanation + Java examples + diagrams

Key points

Array indexes start at 0; length gives the element count; arrays have fixed size; Arrays.sort() sorts; Arrays.equals() compares contents.

Exam tip

Always check the array bounds. For loops commonly use i < array.length, not i <= array.length.

Java code example

```
import java.util.Arrays;
class ArrayRevision {
    public static void main(String[] args) {
        int[] marks = {75, 92, 68, 84};
        Arrays.sort(marks);
        System.out.println(Arrays.toString(marks));
        int pos = Arrays.binarySearch(marks, 84);
        System.out.println("Index = " + pos);
    }
}
```

Quick comparison

Task	Method / syntax
Print array	Arrays.toString(a)
Sort	Arrays.sort(a)
Search	Arrays.binarySearch(a, key)
Compare contents	Arrays.equals(a, b)
Copy	Arrays.copyOf(a, length)